

Roteiro de Apresentação: DevTrack

Este é o guia definitivo para a sua apresentação de 15 minutos, dividido exatamente entre você (Fullstack) e seus amigos (Banco de Dados).

PARTE 1: Você (Frontend e Backend) - Aprox. 8 a 10 minutos

Sua missão é mostrar que o aplicativo é real, rápido, seguro e construído com as melhores práticas de mercado.

1. Abertura e O Problema (1 min)

"Bom dia/boa noite a todos. Nosso projeto é o **Dev Track**. Percebemos que as plataformas de estudo hoje (como LinkedIn ou GitHub) focam muito na validação social. Nós criamos uma solução focada na **disciplina individual**. Inspirados na gamificação do Duolingo, transformamos o estudo diário de programação em um jogo, onde o usuário ganha XP, mantém ofensivas (streaks) e compete num ranking local."

2. O Frontend: Mobile e UI (3 min)

- Mostre o aplicativo rodando (tela de Login, Trilhas, Avatar).*
"No Frontend, construímos o aplicativo usando **React Native com Expo** e **TypeScript**.
O grande diferencial visual do nosso app foi o uso da biblioteca **Reanimated**. Todas as animações, como o carregamento do XP e as transições de tela, rodam a 60 quadros por segundo na thread nativa do aparelho, garantindo uma fluidez de aplicativo premium.
Para navegação, usamos o **Expo Router** (uma arquitetura moderna baseada em arquivos) e para o estado global (como o sistema de vidas e autenticação) aplicamos a **Context API** do React."

3. O Backend: Regras de Negócio e Segurança (4 min)

- Mostre a tela do Swagger rodando no navegador.*
"Toda a inteligência do DevTrack mora no nosso Backend, desenvolvido em **Java 21 com Spring Boot 3**. Nós estruturamos o código seguindo a **Clean Architecture** (Arquitetura Limpa), separando tudo em Controllers (rotas), Use Cases (regras de negócio) e Entities (banco de dados).

Para segurança, implementamos o **Spring Security com JWT (JSON Web Token)**. Nossa API é 100% *stateless*. Isso significa que quando o usuário faz login, ele recebe um token criptografado. Todas as ações dele no app (como registrar um aprendizado) exigem esse token. Se um 'Visitante' tentar acessar o Ranking, o Spring Boot barra a requisição na mesma hora devolvendo um Erro 403 (Proibido), garantindo a integridade dos dados."

PARTE 2: Seus Amigos (Banco de Dados) - Aprox. 5 a 7 minutos

A missão deles é mostrar que o banco não é só "guardar dados", mas sim uma arquitetura pensada para o mobile (Offline) e para o servidor (Relacional).

Fala 1: O Banco de Dados Central (MySQL)

Amigo 1: "Boa noite. Para suportar toda a lógica do servidor que acabou de ser apresentada, nós optamos pelo **MySQL** como nosso banco de dados relacional central.

Nós não criamos as tabelas manualmente. Utilizamos o **Hibernate (JPA)** integrado ao Spring Boot. Isso significa que nosso banco é gerado a partir do código Java (Code-First).

Nossa tabela de Usuários (`app_users`), por exemplo, está mapeada em relacionamentos de **1-para-Muitos** com a tabela de Aprendizados (`learnings`) e **1-para-1** com a tabela de Ofensiva (`streak`). O uso do JPA garante que todas as nossas queries sejam otimizadas e protegidas nativamente contra ataques de *SQL Injection*."

Fala 2: A Estratégia Offline-First (SQLite + AsyncStorage)

Amigo 2: "Porém, estamos lidando com um aplicativo Mobile, e conexões caem. Para resolver isso, nós implementamos uma arquitetura de banco de dados híbrida chamada **Offline-First**.

No próprio celular do usuário, o React Native roda um banco de dados local usando o **SQLite** e o **AsyncStorage**.

Sempre que o usuário tem internet, nós salvamos o perfil, o XP, e as vidas dele no banco local. Se ele entrar em um túnel ou abrir o app no modo avião, o aplicativo intercepta a falha de rede (`catch`) e carrega os dados instantaneamente do SQLite. Assim, ele pode continuar estudando e o sistema de Vidas continua calculando o tempo sem depender do servidor."

PARTE 3: A Banca (Perguntas e Respostas de 15 minutos)

Aqui estão as perguntas que os professores adoram fazer e como vocês devem responder:

? "Por que usar JWT em vez de Sessão comum (Cookies)?"

Você (Backend): "Como temos o Frontend no Mobile e o Backend num servidor separado, o JWT é a melhor escolha porque a API fica *stateless*. O nosso servidor Java não precisa gastar memória lembrando quem está logado. Toda a identidade e autorização do usuário viajam de forma criptografada dentro do próprio token."

? "O que acontece se eu mudar o relógio do celular para ganhar mais Vidas?"

Você (Frontend): "Atualmente, nosso sistema de vidas local usa o relógio do dispositivo (`Date.now()`) para calcular o tempo de regeneração (30 minutos por vida) e permitir que o jogo funcione offline. Numa versão de produção financeira, nós validaríamos a *timestamp* diretamente do servidor assim que o celular voltasse a ter internet."

? "Como vocês garantem que a senha do usuário não vai ser roubada no banco?"

Amigos (Banco de Dados): "No nosso banco MySQL, nós nunca salvamos a senha em texto puro. Utilizamos o algoritmo de hash **BCrypt** dentro do Spring Security. Ele gera um código irreversível. Nem nós, como desenvolvedores, sabemos a senha dos usuários."

? "E se houver muitos usuários, o Ranking não vai deixar o servidor lento?"

Amigos (Banco de Dados): "O ranking é ordenado por XP ou Streak. No backend, a nossa consulta para o Ranking já utiliza os recursos do Spring Data JPA com paginação (limitando a 50 usuários por vez). Num cenário futuro com milhares de usuários, criaríamos **Índices (Indexes)** na coluna `xp` e `streak` no MySQL para garantir que a leitura seja em frações de segundo."

? "Onde está a Inteligência Artificial que foi proposta?"

Você (Fullstack): "A arquitetura para a IA (usando o Google Gemini) já está escrita no nosso código mobile (`ai.service.ts`), com o objetivo de dar feedback nos textos que os usuários digitam. Porém, para a apresentação de hoje em ambiente localhost, e por questões de segurança de chaves de API, nós focamos 100% em consolidar nossa API Spring Boot e o MySQL, entregando um sistema gamificado robusto primeiro."